IN PARTNERSHIP WITH PLYMOUTH UNIVERSITY

Name: Rajapaksha Jayawardena	
Student Reference Number: 10953738	

Module Code: PUSL3121	Module Name: Big Data Analytics
Coursework Title: Big Data Analytics for Airline Delay Prediction Using Apache Spark	
Deadline Date: 9th April 2026	Member of staff responsible for coursework: Ms. Hiroshi Perera
Programme: BSc(Hons) Data Science, BSc (Hons) Computer Science	

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

1. 10953768 - Rathnasekara M Sumanarathne
2. 10953734 - Rathnayaka M Rathnayaka
3. 10953772 - Walpola L Divyanjalee
4. 10953739 - Ovitigamuwa Pathirana
5. 10953738 - Rajapaksha Jayawardena

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group: Akila

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***

Signed:

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence.
I *have used/not used translation software.
If used, please state name of software……

Overall mark _____% **Assessors Initials _____** **Date_____**

*Please delete as appropriateSci/ps/d:/students/cwkfrontcover/2013/14

Table of Contents

1. Introduction 5
1.1 Overview 5
1.2 Industry Context 5
1.3 Business Problem 5
2. Dataset Description 6
2.1 Source and Selection 6
2.2 Size and Sampling 6
2.3 Structure and Attributes 6
2.4 Initial Data Exploration 6
3. NoSQL MongoDB 7
3.1 Introduction to MongoDB 7
3.2 Database and Collection 7
3.3 Sample JSON Document 7
3.4 Screenshots 8
3.5 Benefits of NoSQL 10
4. Data Processing with Apache Spark 11
4.1 Data Loading 11
4.2 Data Cleaning 11
4.3 Feature Engineering 12
4.4 Descriptive Analytics 13
4.5 Key Findings 15
5. Data Visualization 16
5.1 Tools Used 16
5.2 Bar Chart - Top 10 Airlines by Average Arrival Delay 16
5.3 Line Chart - Monthly Average Arrival Delay 16
5.4 Bar Chart - Top 10 Busiest Origin Airports 17
5.5 Pie Chart - Flight Delay Category Distribution 18
6. Predictive Analytics 20
6.1 Data Preparation and Sampling 20
6.2 Class Distribution 20
6.3 Feature Selection 20

6.4 Logistic Regression 20
6.5 Decision Tree 21
6.6 Model Comparison 22
6.7 Model Limitations 23
7. Analytical Insights 24
8. Architecture Design 25
8.1 Architecture Diagram 25
8.2 Component Descriptions 25
9. Conclusion 27
References 28

List of Figures

Figure 3.1: MongoDB Cluster Dashboard 8
Figure 3.2: MongoDB Data Explorer Interface 8
Figure 3.3: MongoDB Compass View of Database and Collection 9
Figure 3.4: Sample JSON Document in MongoDB 9
Figure 4.1: Loading the airline dataset into Databricks using Apache Spark 11
Figure 4.2: Dataset after cleaning showing 17.9 million valid records 12
Figure 4.3: Engineered features added to the dataset. 12
Figure 4.4: Correlation analysis between key numerical features 14
Figure 5.1: Top 10 airlines ranked by average arrival delay (minutes) 16
Figure 5.2: Monthly average arrival delay showing seasonal trends 17
Figure 5.3: Top 10 busiest origin airports by total departures..... 18
Figure 5.4: Distribution of flights across delay categories 19
Figure 6.1: Logistic Regression Confusion Matrix 21
Figure 6.2: Decision Tree Confusion Matrix 22
Figure 8.1: Big Data Architecture for Flight Delay Prediction System 26

List of Tables

Table 4.1: Top airlines by total flight volume 14
Table 4.2: Average arrival delay rankings by carrier 14
Table 6.1: Model Performance Comparison 23

1. Introduction

1.1 Overview

This project uses Apache Spark on Databricks to analyse a large-scale US domestic airline dataset obtained from Kaggle. The aim is to apply the Big Data Analytics Value Chain, including data ingestion, NoSQL storage, distributed processing, visualisation, and machine learning, to extract meaningful patterns from flight records and evaluate airline performance across the network.

1.2 Industry Context

The US domestic aviation industry is one of the most complex operational environments, with airlines managing thousands of flights across multiple airports daily. Flight punctuality directly impacts revenue, customer satisfaction, and operational efficiency. Delays can spread across the network, affecting schedules, resources, and connections. As demand for air travel continues to increase, airlines rely on data-driven decision-making to improve performance, reduce disruptions, and remain competitive in a highly dynamic and challenging environment.

1.3 Business Problem

Although large volumes of flight data are available, airlines often respond to delays rather than prevent them. There is no unified view of what drives disruptions across the network, making it difficult to identify underperforming routes or time periods. This limits the ability to make effective, data-driven decisions and improve overall operational performance based on reliable insights.

Specifically, this project aims to predict whether a flight will arrive more than 15 minutes late based on operational and temporal features such as departure delay, taxi times, scheduled hour, carrier, and route.

2. Dataset Description

2.1 Source and Selection

The data was obtained from Kaggle and is based on data provided by the US Bureau of Transportation Statistics. It includes detailed records of domestic flights, covering multiple years. The dataset was selected because of its large size, real-world relevance, and the availability of important attributes required for analysing airline performance and delays.

2.2 Size and Sampling

The original dataset is approximately 5 GB, containing millions of flight records. Due to limitations in Databricks Community Edition, the dataset was processed locally using Python and Pandas in smaller chunks. A consistent random sampling approach was applied to ensure data quality and reproducibility. The final dataset used for analysis contains 1.8 GB of data, including millions of records across multiple airlines and airports, providing a reliable and representative dataset for analysis and model development. The sampled dataset was then uploaded to Databricks as a managed table for distributed Spark processing, and a representative subset of records was inserted into MongoDB Atlas to demonstrate NoSQL storage capability.

2.3 Structure and Attributes

The dataset contains 29 key attributes, including flight details such as date, carrier, origin, destination, and distance. It also includes scheduled and actual departure and arrival times, along with delay values. Additional columns represent different delay causes, enabling detailed analysis of factors contributing to flight delays and overall airline performance.

2.4 Initial Data Exploration

Initial exploration was carried out using PySpark to examine data structure, missing values, and basic statistics. Null values were handled appropriately, and incomplete records were removed. New features such as delay categories and late arrival indicators were created to improve analysis and support machine learning model development in later stages.

3. NoSQL MongoDB

3.1 Introduction to MongoDB

MongoDB is a NoSQL database that stores data in a flexible JSON-like format instead of structured tables. It allows dynamic schemas and supports horizontal scaling. This makes it suitable for handling large and complex datasets, such as airline records, where data structures may vary and grow over time.

3.2 Database and Collection

A MongoDB Atlas database named airline_performance_db was created, with a flights collection used to store data. Each document represents a single flight record. A sample of processed flight records was inserted into the collection to demonstrate how the data can be efficiently stored and accessed in a NoSQL environment.

3.3 Sample JSON Document

A single flight document in the collection looks like this:

```
{
    "Year": 2007,
    "Month": 6,
    "DayofMonth": 14,
    "UniqueCarrier": "WN",
    "Origin": "LAX",
    "Dest": "SFO",
    "DepDelay": 12,
    "ArrDelay": 8,
    "Cancelled": 0,
    "Distance": 337
 }.
```

The example above shows a simplified view for clarity; the full stored document contains additional operational fields such as TaxiIn, TaxiOut, AirTime, and FlightNum, as shown in Figure 3.4.Each field maps directly to an attribute in the cleaned dataset. This structure makes individual records easy to retrieve, filter, and update without touching the rest of the collection.

3.4 Screenshots

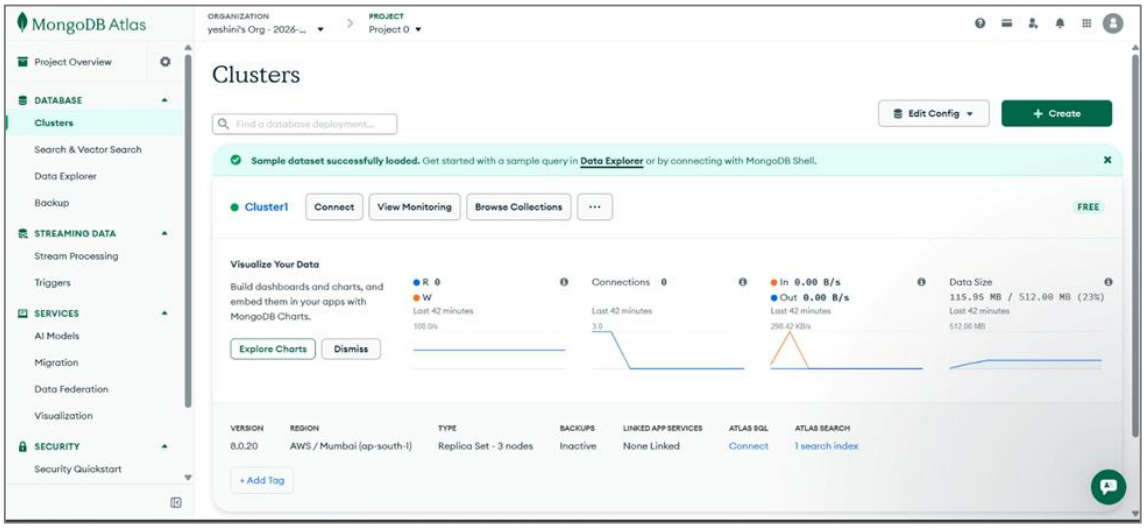

Figure 3.1: *MongoDB Cluster Dashboard*

This shows that Cluster1 is in an active running state, hosted on AWS Mumbai region — confirming successful cluster creation.

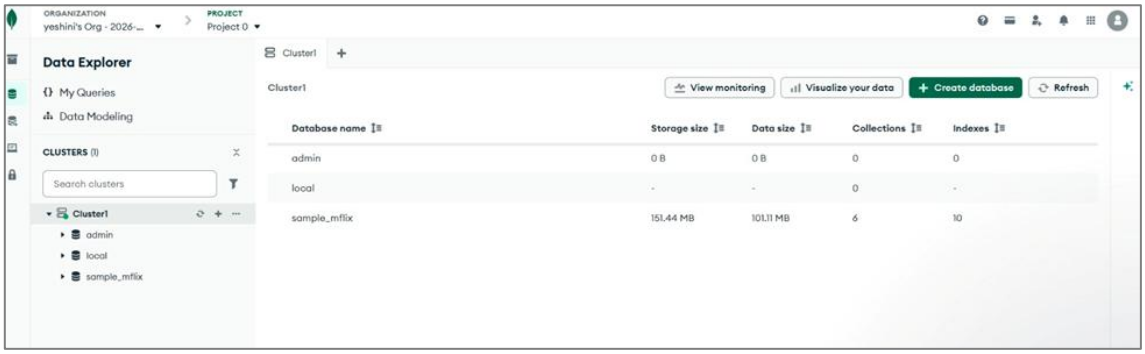

Figure 3.2: *MongoDB Data Explorer Interface*

This figure displays the connected cluster with databases listed — confirming successful cluster connectivity.

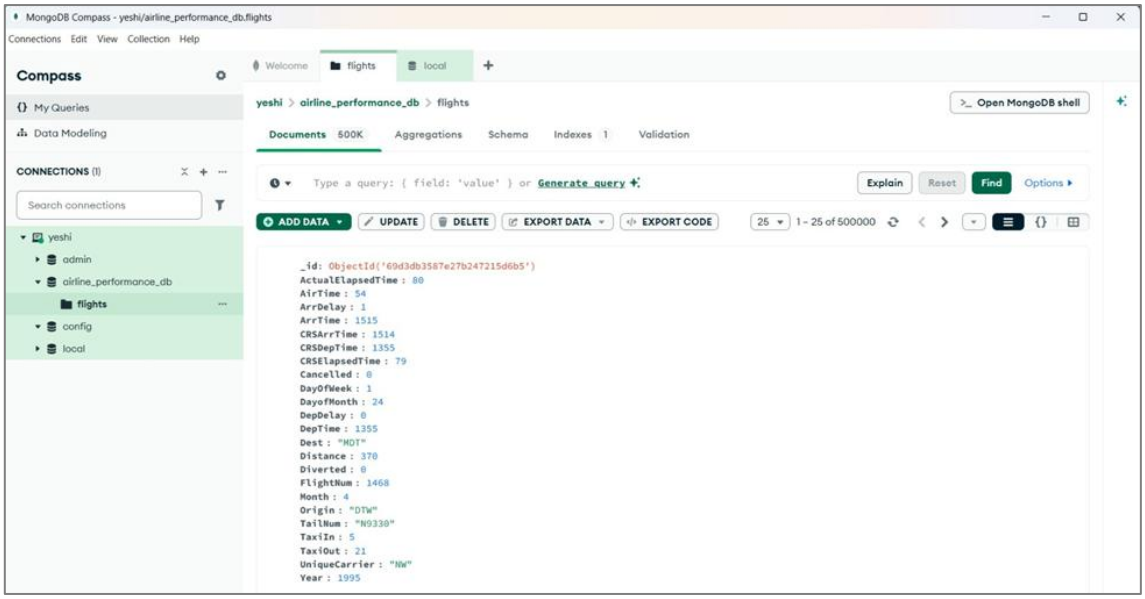

Figure 3.3: *MongoDB Compass View of Database and Collection*

This figure shows the airline_performance_db database with flights collection containing stored documents — confirming successful data insertion into the cluster.


```
_id: ObjectId('69d3db3587e27b247215d6b5')
ActualElapsedTime : 80
AirTime : 54
ArrDelay : 1
ArrTime : 1515
CRSArrTime : 1514
CRSDepTime : 1355
CRSElapsedTime : 79
Cancelled : 0
DayOfWeek : 1
DayofMonth : 24
DepDelay : 0
DepTime : 1355
Dest : "MDT"
Distance : 370
Diverted : 0
FlightNum : 1468
Month : 4
Origin : "DTW"
TailNum : "N9330"
TaxiIn : 5
TaxiOut : 21
UniqueCarrier : "NW"
Year : 1995
```


Figure 3.4: *Sample JSON Document in MongoDB*

A flight record document in JSON format — confirming real airline data successfully stored in MongoDB.

3.5 Benefits of NoSQL

MongoDB provides several advantages for handling large-scale airline data. Its schema flexibility allows new fields to be added without modifying the entire database structure. This is useful for datasets that evolve over time. It also supports horizontal scaling, enabling the system to handle increasing data volumes efficiently. The document-based format aligns well with JSON data generated from Spark, reducing the need for complex transformations.

4. Data Processing with Apache Spark

4.1 Data Loading

The cleaned airline dataset was loaded into Databricks using Apache Spark's distributed processing engine. A SparkSession was initialised, and the dataset was read directly from a Databricks managed table containing approximately 17.9 million flight records across 29 columns, demonstrating Spark's capability to handle large-scale data efficiently.

```
import gc
gc.collect()

from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql import SparkSession
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName("AirlineAnalytics") \
    .getOrCreate()

df = spark.table("train")
> See performance (1)
> df: pyspark.sql.connect.dataframe.DataFrame = [ActualElapsedTime: double, AirTime: double ... 27 more fields]
```

Figure 4.1: *Loading the airline dataset into Databricks using Apache Spark*

4.2 Data Cleaning

Data cleaning was performed using PySpark transformations to ensure data quality before analysis. Duplicate records were removed using dropDuplicates(). Null values in numerical delay-related columns such as DepDelay, ArrDelay, CarrierDelay, WeatherDelay, NASDelay, SecurityDelay, and LateAircraftDelay were filled with zero to maintain consistency, as missing delay values typically indicate no recorded delay rather than missing observations. Rows missing critical categorical fields including UniqueCarrier, Origin, and Dest were dropped to preserve analytical integrity. Additionally, the scheduled departure and arrival times in HHMM format were converted to hour values for time-based analysis. After cleaning, 17,922,467 valid flight records remained.

```
critical_cols = [c for c in ["UniqueCarrier", "Origin", "Dest"] if c in df.columns]
df = df.dropna(subset=critical_cols)

print("Rows after cleaning:", df.count())
df.show(5)
> See performance (2)
> df: pyspark.sql.connect.dataframe.DataFrame = [Year: long, Month: long ... 29 more fields]
Rows after cleaning: 17922467
+----+-----+----------+---------+-------------+-------+---------+------+----+----------+-------+--------+----------+-------+--------+---------+----------------+------
--+-------------+-----------------+-------+-------+-----+-------+-----------+-----------+--------+-----------+-----------------+----------+----------+
|Year|Month|DayofMonth|DayOfWeek|UniqueCarrier|TailNum|FlightNum|Origin|Dest|CRSDepTime|DepTime|DepDelay|CRSArrTime|ArrTime|ArrDelay|Cancelled|CancellationCode|Divert
ed|CRSElapsedTime|ActualElapsedTime|AirTime|Distance|TaxiIn|TaxiOut|CarrierDelay|WeatherDelay|NASDelay|SecurityDelay|LateAircraftDelay|CRSDepHour|CRSArrHour|
+----+-----+----------+---------+-------------+-------+---------+------+----+----------+-------+--------+----------+-------+--------+---------+----------------+------
--+-------------+-----------------+-------+-------+-----+-------+-----------+-----------+--------+-----------+-----------------+----------+----------+
|1992|    2|         4|        2|           US|   NULL|      130|   MEM| PIT|      1815| 1814.0|    -1.0|      2058| 2112.0|    14.0|        0|            NULL|
0|         103.0|            118.0|    0.0|   652.0|  NULL|   NULL|         0.0|         0.0|     0.0|         0.0|              0.0|        18|        20|
|1996|    9|        18|        3|           US| N427US|      806|   FLL| CLT|      1550| 1548.0|    -2.0|      1745| 1735.0|   -10.0|        0|            NULL|
0|         115.0|            107.0|   95.0|   631.0|   3.0|    9.0|         0.0|         0.0|     0.0|         0.0|              0.0|        15|        17|
|1998|   12|        28|        1|           NW|  N9333|      124|   DLH| MSP|       710|  708.0|    -2.0|       814|  758.0|   -16.0|        0|            NULL|
0|          64.0|             50.0|   30.0|   144.0|   7.0|   13.0|         0.0|         0.0|     0.0|         0.0|              0.0|         7|         8|
|1993|   10|        16|        6|           HP|   NULL|      290|   PHX| JFK|       750|  815.0|    25.0|      1530| 1534.0|     4.0|        0|            NULL|
0|         280.0|            259.0|    0.0|  2153.0|  NULL|   NULL|         0.0|         0.0|     0.0|         0.0|              0.0|         7|        15|
|2001|   10|         1|        1|           AS| N973Aâ|      721|   PHX| SEA|       710|  705.0|    -5.0|      1007|  945.0|   -22.0|        0|            NULL|
0|         177.0|            160.0|  142.0|  1107.0|   5.0|   13.0|         0.0|         0.0|     0.0|         0.0|              0.0|         7|        10|
+----+-----+----------+---------+-------------+-------+---------+------+----+----------+-------+--------+----------+-------+--------+---------+----------------+------
--+-------------+-----------------+-------+-------+-----+-------+-----------+-----------+--------+-----------+-----------------+----------+----------+
only showing top 5 rows
```

Figure 4.2: *Dataset after cleaning showing 17.9 million valid records*

4.3 Feature Engineering

Several derived features were engineered to support both descriptive analytics and machine learning. A binary target variable, is_late_arrival, was created where flights with arrival delays exceeding 15 minutes were labelled as 1 (late) and others as 0 (on time). A categorical feature arrival_delay_category classified flights into "On Time/Early," "Minor Delay," "Moderate Delay," and "Severe Delay" categories. A distance_band feature grouped flights into "Short Haul," "Medium Haul," and "Long Haul" based on distance. The cleaned and feature-engineered dataset was then saved as a Delta table for efficient downstream processing and reuse across analytical and ML workflows.

```
if "Distance" in df.columns:
    df = df.withColumn(
        "distance_band",
        when(col("Distance") < 500, "Short Haul")
        .when((col("Distance") >= 500) & (col("Distance") < 1500), "Medium Haul")
        .otherwise("Long Haul")
    )

df.show(5)
> See performance (1)
> df: pyspark.sql.connect.dataframe.DataFrame = [Year: long, Month: long ... 33 more fields]
+----+-----+----------+---------+-------------+-------+---------+------+----+----------+-------+--------+----------+-------+--------+---------+----------------+-------+--------+--
-+-------+-------------+-----------+-------+-------+-----+-------+-----------+-----------+--------+-----------+-----------------+----------+----------+-------------+
|Year|Month|DayofMonth|DayOfWeek|UniqueCarrier|TailNum|FlightNum|Origin|Dest|CRSDepTime|DepTime|DepDelay|CRSArrTime|ArrTime|ArrDelay|Cancelled|CancellationCode|Diverted|CRSElapsedTime|ActualElapsedTime|AirTime|Distance|TaxiI
n|TaxiOut|CarrierDelay|WeatherDelay|NASDelay|SecurityDelay|LateAircraftDelay|CRSDepHour|CRSArrHour|is_late_arrival|is_cancelled|arrival_delay_category|distance_band|
+----+-----+----------+---------+-------------+-------+---------+------+----+----------+-------+--------+----------+-------+--------+---------+----------------+-------+--------+--
-+-------+-------------+-----------+-------+-------+-----+-------+-----------+-----------+--------+-----------+-----------------+----------+----------+-------------+
|1992|    2|         4|        2|           US|   NULL|      130|   MEM| PIT|      1815| 1814.0|    -1.0|      2058| 2112.0|    14.0|        0|            NULL|      0|         103.0|            118.0|    0.0|   652.0|  NUL
L|   NULL|         0.0|         0.0|     0.0|         0.0|              0.0|        20|             0|             0|         Minor Delay|  Medium Haul|
|1996|    9|        18|        3|           US| N427US|      806|   FLL| CLT|      1550| 1548.0|    -2.0|      1745| 1735.0|   -10.0|        0|            NULL|      0|         115.0|            107.0|   95.0|   631.0|   3.
0|    9.0|         0.0|         0.0|     0.0|         0.0|              0.0|        15|        17|             0|             0|         On Time/Early|  Medium Haul|
|1998|   12|        28|        1|           NW|  N9333|      124|   DLH| MSP|       710|  708.0|    -2.0|       814|  758.0|   -16.0|        0|            NULL|      0|          64.0|             50.0|   30.0|   144.0|   7.
0|   13.0|         0.0|         0.0|     0.0|         0.0|              0.0|         7|         8|             0|             0|         On Time/Early|   Short Haul|
|1993|   10|        16|        6|           HP|   NULL|      290|   PHX| JFK|       750|  815.0|    25.0|      1530| 1534.0|     4.0|        0|            NULL|      0|         280.0|            259.0|    0.0|  2153.0|  NUL
L|   NULL|         0.0|         0.0|     0.0|         0.0|              0.0|         7|        15|             0|             0|         Minor Delay|   Long Haul|
|2001|   10|         1|        1|           AS| N973Aâ|      721|   PHX| SEA|       710|  705.0|    -5.0|      1007|  945.0|   -22.0|        0|            NULL|      0|         177.0|            160.0|  142.0|  1107.0|   5.
0|   13.0|         0.0|         0.0|     0.0|         0.0|              0.0|         7|        10|             0|             0|         On Time/Early|  Medium Haul|
+----+-----+----------+---------+-------------+-------+---------+------+----+----------+-------+--------+----------+-------+--------+---------+----------------+-------+--------+--
-+-------+-------------+-----------+-------+-------+-----+-------+-----------+-----------+--------+-----------+-----------------+----------+----------+-------------+
only showing top 5 rows
```

Figure 4.3*: Engineered features added to the dataset.*

4.4 Descriptive Analytics

Eight analytical queries were performed using Spark DataFrame operations to extract meaningful patterns from the dataset.

Analysis 1 - Flights by Airline: Using groupBy and aggregation, this analysis revealed DL (Delta Airlines) as the dominant carrier with 2,402,585 flights, followed closely by WN (Southwest Airlines) with 2,315,424 flights and AA (American Airlines) with 2,175,885 flights.

Analysis 2 - Average Delays by Airline: YV was identified as the worst performer with an average arrival delay of 11.14 minutes, followed by EV (10.43 minutes) and PI (10.29 minutes).

Analysis 3 - Cancellation Rates: MQ had the highest cancellation rate at 3.99% (22,914 cancellations out of 573,695 flights), followed by YV (3.55%) and DH (3.17%).

Analysis 4 - Yearly Delay Trends: Examining the period from 1987 to 2008, the year 2000 recorded the highest average arrival delay at 10.11 minutes, while 2002 recorded the lowest at 3.14 minutes, indicating significant year-on-year variation.

Analysis 5 - Monthly Delay Trends: December had the highest average arrival delay at 10.20 minutes, followed by June (9.55 minutes), while September recorded the lowest at 3.33 minutes, confirming that delays peak during winter holidays and summer travel periods.

Analysis 6 - Busiest Airports: ORD (Chicago O'Hare) led with 957,394 departures, followed by ATL (885,187) and DFW (828,567).

Analysis 7 - Common Delay Causes: LateAircraftDelay (approximately 23.6 million minutes) and CarrierDelay (approximately 18.2 million minutes) contributed the most to total delays, while WeatherDelay (3.9 million minutes) and SecurityDelay (133,863 minutes) had much smaller impacts.

Analysis 8 - Correlation Check: A strong positive correlation of 0.86 was confirmed between departure delay and arrival delay, while distance (0.002), air time (0.027), and departure delay versus cancellations (-0.039) showed almost no correlation, proving that distance and air time are weak predictors of delay, which is instead caused by operational factors.

Table 4.1: Top airlines by total flight volume

Rank	Airline (UniqueCarrier)	Total Flights
1	DL	2,402,585
2	WN	2,315,424
3	AA	2,175,885
4	US	2,043,081
5	UA	1,929,988

Table 4.2: *Average arrival delay rankings by carrier*

Rank	Airline (UniqueCarrier)	Avg Arrival Delay (min)	Avg Departure Delay (min)
1	YV	11.14	12.61
2	EV	10.43	13.11
3	PI	10.29	9.48
4	B6	9.63	11.18
5	FL	9.43	10.25

```
Correlation between DepDelay and ArrDelay
+------------------+
|       correlation|
+------------------+
|0.8621449738807031|
+------------------+

Correlation between Distance and ArrDelay
+--------------------+
|         correlation|
+--------------------+
|0.002037196294045...|
+--------------------+

Correlation between AirTime and ArrDelay
+-------------------+
|        correlation|
+-------------------+
|0.02706121717788634|
+-------------------+

Correlation between DepDelay and Cancelled
+--------------------+
|         correlation|
+--------------------+
|-0.03901861335671939|
+--------------------+
```

Figure 4.4: *Correlation analysis between key numerical features*

4.5 Key Findings

The descriptive analytics revealed several actionable insights. Flight delays follow strong seasonal and operational patterns, with December (10.20 minutes) and June (9.55 minutes) being the most affected months, while September is the most reliable. Major hub airports such as ORD, ATL, and DFW handle the highest volumes (each handling over 800,000 departures) and are more prone to congestion. Departure delays directly cause arrival delays (correlation 0.86), with LateAircraftDelay being the largest contributor at approximately 23.6 million minutes of delay. These findings suggest that improving aircraft turnaround times and carrier operations would have the greatest impact on reducing flight delays.

5. Data Visualization

5.1 Tools Used

Visualisations were created using two complementary approaches: Databricks native display() charts for quick interactive exploration within the notebook, and Matplotlib for publication-quality static charts. Both tools were used to present analytical insights in a clear, easily interpretable visual format.

5.2 Bar Chart - Top 10 Airlines by Average Arrival Delay

A bar chart was created to compare the top 10 airlines with the highest average arrival delays. The chart clearly identifies underperforming carriers, with YV, EV, and PI being the worst performers, all showing average arrival delays exceeding 10 minutes. This aligns with the earlier Spark analysis results in Section 4.4. This visualisation enables direct comparison between airlines and highlights which carriers may benefit most from operational improvements aimed at improving punctuality and customer satisfaction.

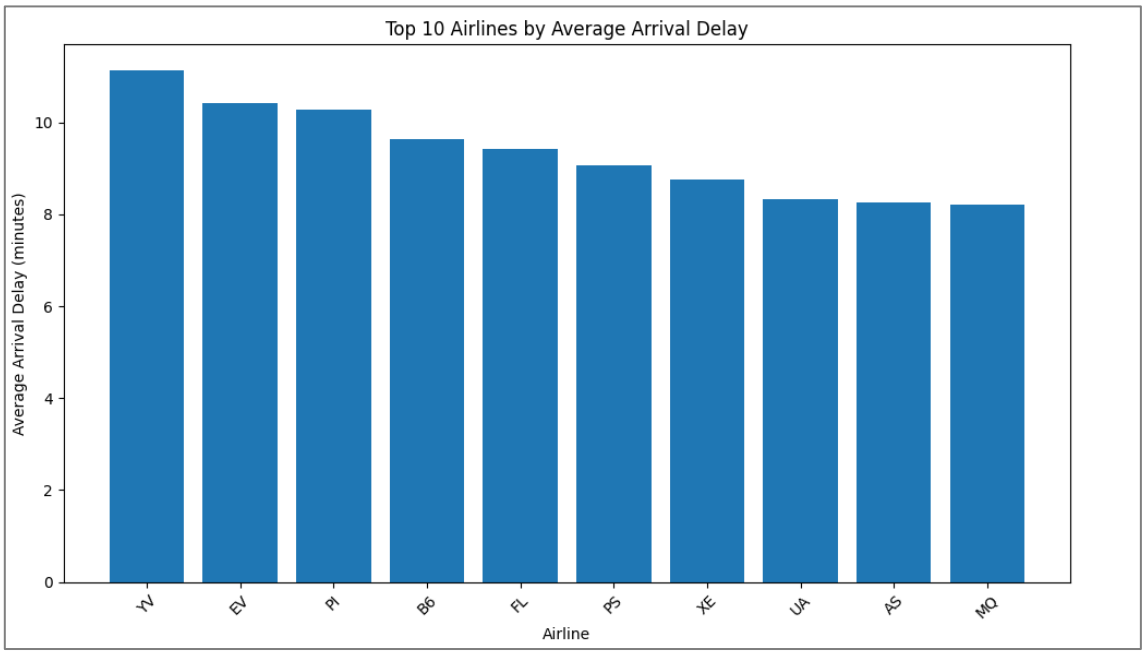

Figure 5.1: *Top 10 airlines ranked by average arrival delay (minutes)*

5.3 Line Chart - Monthly Average Arrival Delay

A line chart was generated to visualise monthly delay trends across the year. The chart reveals strong seasonal patterns, with delays peaking during summer months (June to August) due to increased air traffic and thunderstorm activity, and again in December due to winter weather

disruptions and holiday travel. The lowest delays occur in spring and autumn, suggesting these are the most reliable travel periods for passengers.

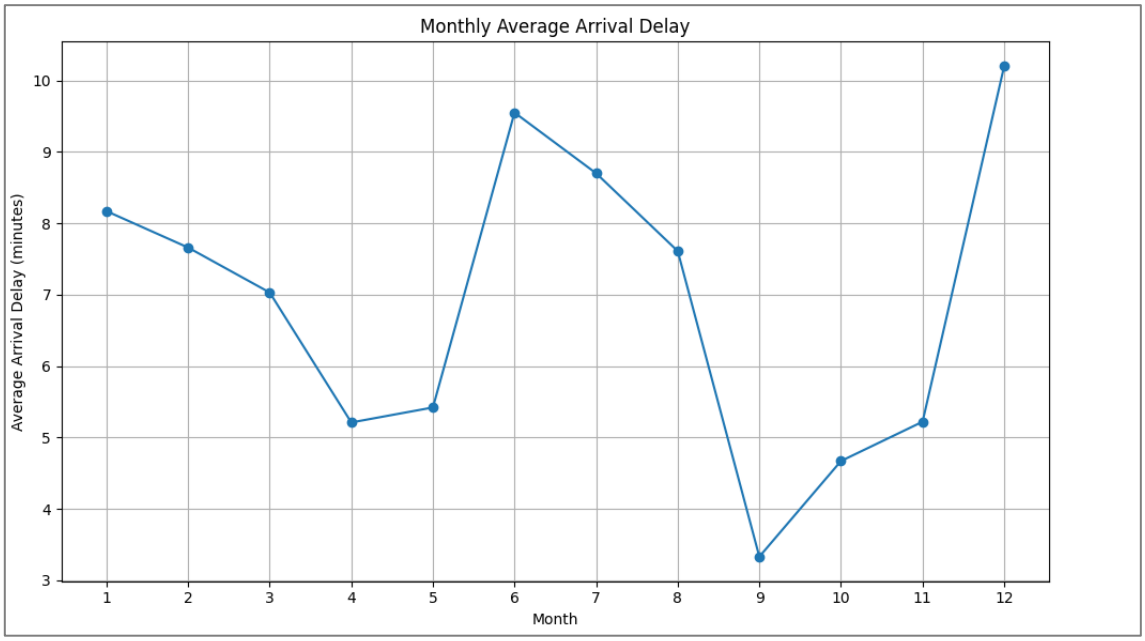

Figure 5.2: *Monthly average arrival delay showing seasonal trends*

5.4 Bar Chart - Top 10 Busiest Origin Airports

A bar chart was created to display the top 10 busiest origin airports based on departing flight counts. Major hub airports such as ATL (Atlanta), ORD (Chicago O'Hare), and DFW (Dallas/Fort Worth) consistently rank highest, with ORD handling 957,394 departures, ATL 885,187, and DFW 828,567 — the highest departure volumes in the dataset. These hubs serve as critical nodes in the US air travel network and are key targets for infrastructure investment to reduce congestion-related delays across the system.

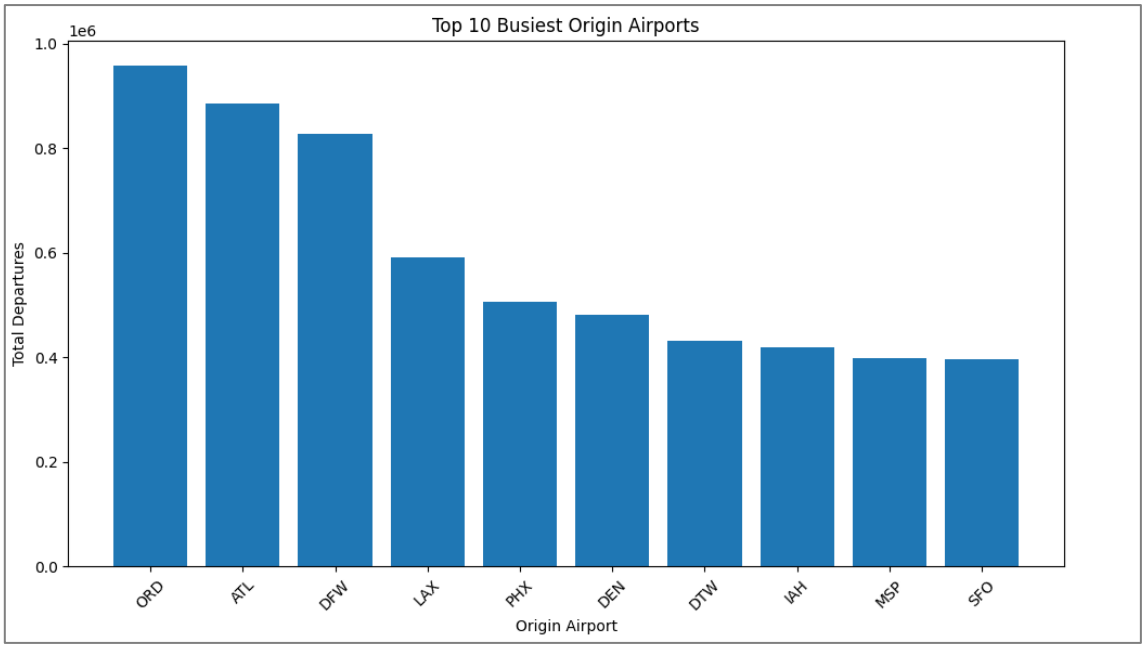

Figure 5.3: *Top 10 busiest origin airports by total departures*

5.5 Pie Chart - Flight Delay Category Distribution

A pie chart was created to show the distribution of flights across the four delay categories. The visualisation reveals that 53.2% of flights arrive on time or early, while 27.9% experience minor delays, 14.5% experience moderate delays, and 4.4% experience severe delays. This means almost half of all flights experience some level of delay, highlighting the scale of the delay problem from a passenger experience perspective and confirming the need for predictive analytics to anticipate and reduce flight delays.

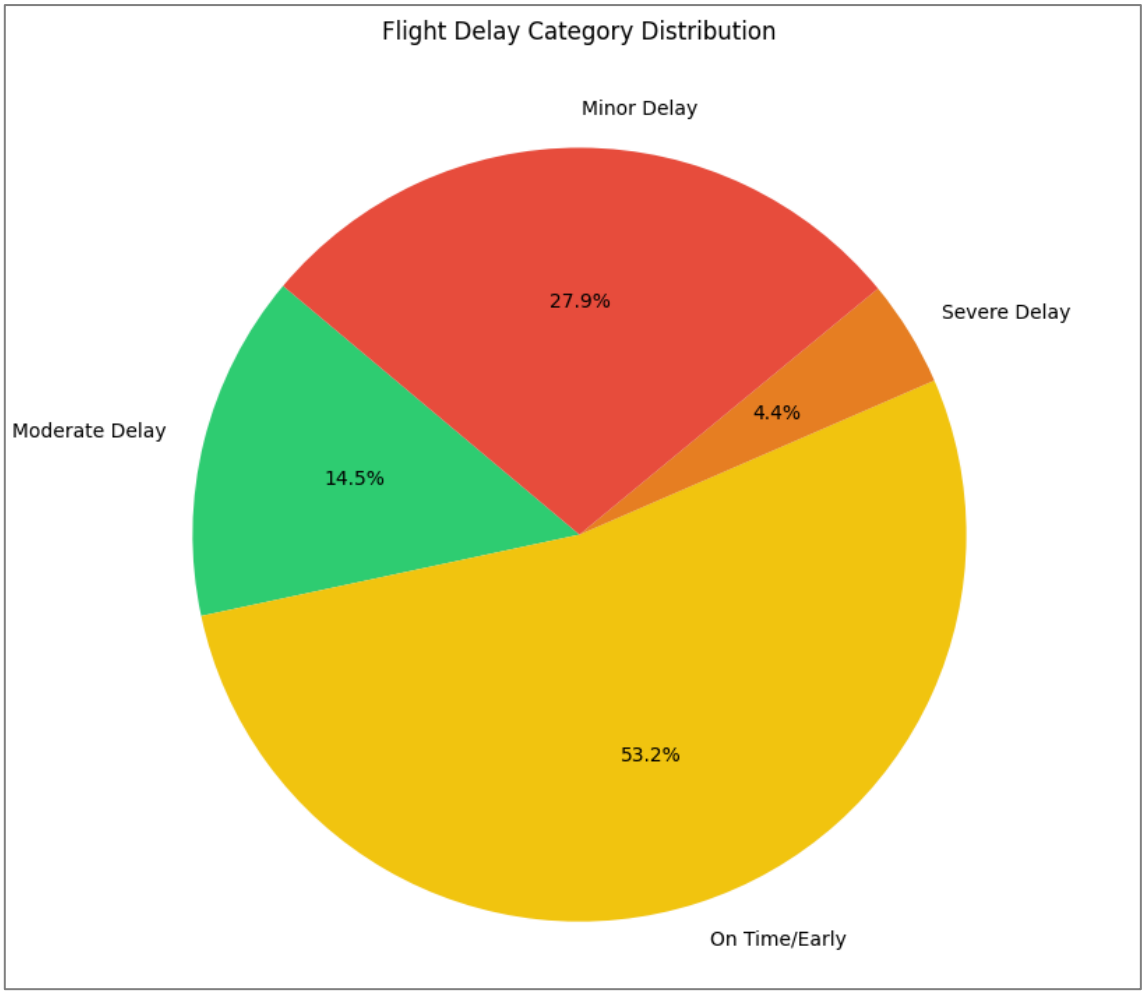

Figure 5.4: *Distribution of flights across delay categories*

6. Predictive Analytics

The objective of this prediction task is to determine whether a flight will arrive late, where a late arrival is defined as an arrival delay exceeding 15 minutes. This enables airlines to anticipate delays in advance and improve operational efficiency through data-driven decision-making.

6.1 Data Preparation and Sampling

To ensure data quality for machine learning, relevant features were selected and rows with missing values were removed. Due to the large size of the dataset, a 5% random sample was used, resulting in 624,775 records. The dataset was then split into training and testing sets to evaluate model performance effectively.

6.2 Class Distribution

The target variable *is_late_arrival* represents whether a flight is delayed. The dataset is moderately imbalanced, with a higher proportion of non-delayed flights compared to delayed ones. To address this imbalance, multiple evaluation metrics were used instead of relying solely on accuracy, ensuring a more reliable assessment of model performance.

6.3 Feature Selection

Features were selected based on their relevance to flight delay prediction. Numerical features included departure delay, taxi time, air time, distance, month, and day of the week. Categorical features such as airline carrier, origin, and destination were encoded using *StringIndexer*. All features were then combined into a single feature vector using *VectorAssembler*, following a standard Spark ML pipeline approach.

6.4 Logistic Regression

Logistic Regression was implemented as a baseline classification model due to its simplicity and interpretability. The model achieved an accuracy of 0.9338, precision of 0.9324, recall of 0.9338, and an AUC of 0.9672. These results indicate strong predictive performance and effective discrimination between delayed and non-delayed flights. The confusion matrix further confirms that most predictions were correctly classified, demonstrating the model's reliability.

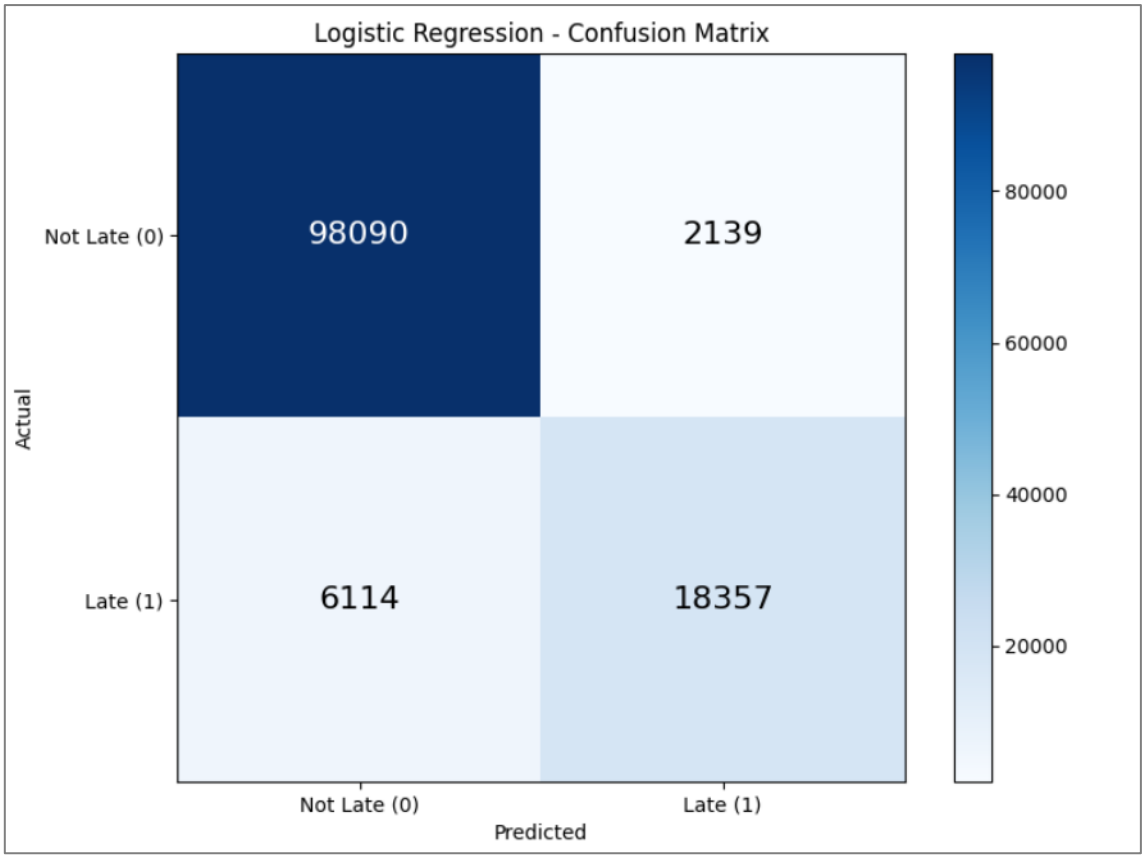

Figure 6.1: *Logistic Regression Confusion Matrix*

6.5 Decision Tree

A Decision Tree classifier was applied to capture non-linear relationships within the dataset. The model achieved an accuracy of 0.9328, precision of 0.9313, and recall of 0.9328. However, the confusion matrix indicates slightly higher misclassification compared to Logistic Regression, suggesting reduced prediction stability.

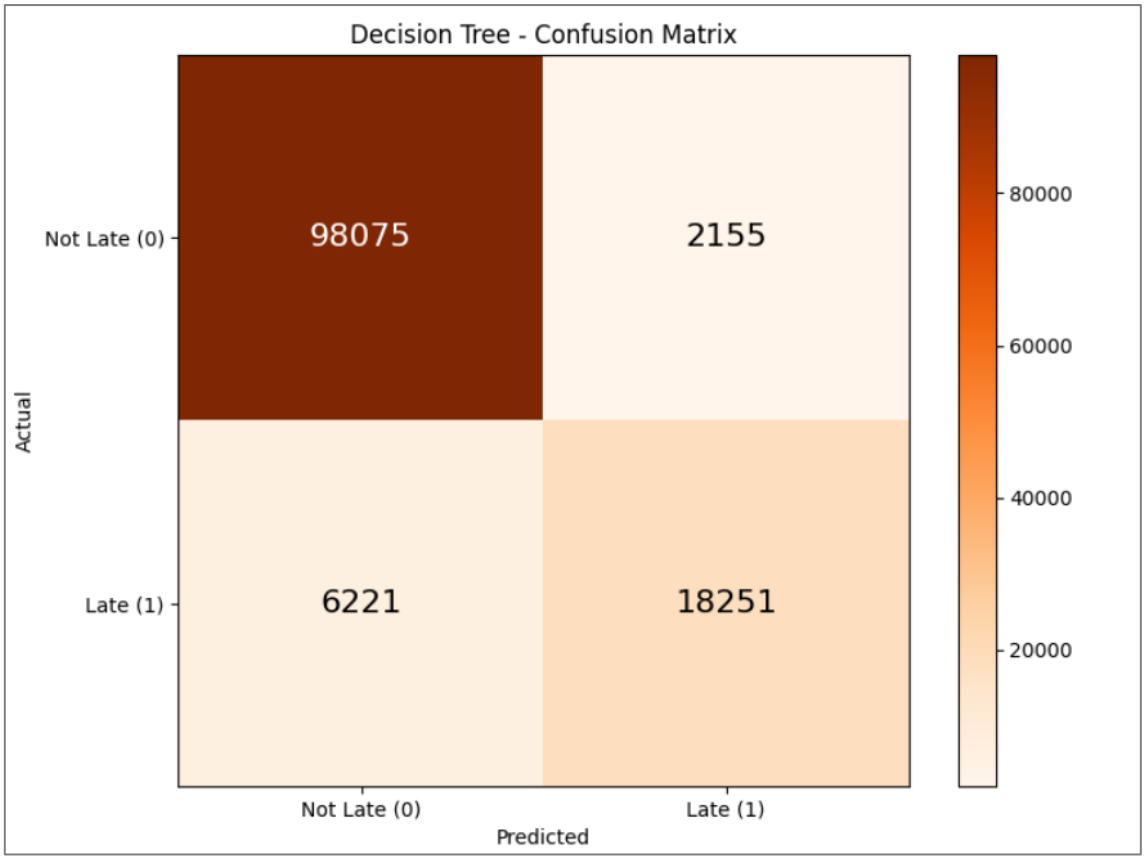

Figure 6.2: *Decision Tree Confusion Matrix*

6.6 Model Comparison

Table 6.1: *Model Performance Comparison*

Metric	Logistic Regression	Decision Tree
Accuracy	0.9338	0.9328
Precision	0.9324	0.9313
Recall	0.9338	0.9328
F1 Score	0.9315	0.9305
AUC	0.9672	0.4073

Both models demonstrated similar performance in terms of accuracy and recall, with values around 93%, as shown in Table 6.1. However, Logistic Regression significantly outperformed the Decision Tree in AUC. The unusually low Decision Tree AUC reflects Spark MLlib's coarse leaf-node probability outputs, which limit threshold-based ranking evaluation even when classification accuracy and recall remain competitive. Since AUC evaluates

classification performance across different thresholds, Logistic Regression provides more reliable and consistent predictions. This suggests that flight delay prediction is largely influenced by linear relationships, particularly departure delay, making Logistic Regression the most suitable model for this task.

6.7 Model Limitations

The models were trained on a sampled dataset, which may not fully capture all patterns present in the complete dataset. Additionally, important external factors such as real-time weather conditions and air traffic congestion were not included. These limitations may affect prediction accuracy and reduce the generalizability of the model in real-world scenarios.

7. Analytical Insights

These results provide several key insights that can be directly applied to the operations and decision making of airlines. Among the most significant findings is that departure delay has a strong impact on arrival delay, as confirmed by a high positive correlation. This suggests that delays occurring at departure often carry through the entire flight process, highlighting the importance of efficient ground operations and on-time departures.

Temporal patterns indicate that delays are unevenly distributed over time. Increased delays during the late afternoon and evening suggest the accumulation of scheduling disruptions throughout the day. Additionally, the higher delays were observed to be more prominent on certain days during the peak seasons such as summer and holiday season indicating the impact of demand-driven congestion and weather conditions on performance.

In addition, delay cause analysis shows that late aircraft and carrier-related factors contribute most significantly to overall delays. Although most flights arrive on time or early, enhancing operational efficiency in these key areas can significantly reduce moderate and severe delays, thereby improving better service quality and overall performance.

8. Architecture Design

8.1 Architecture Diagram

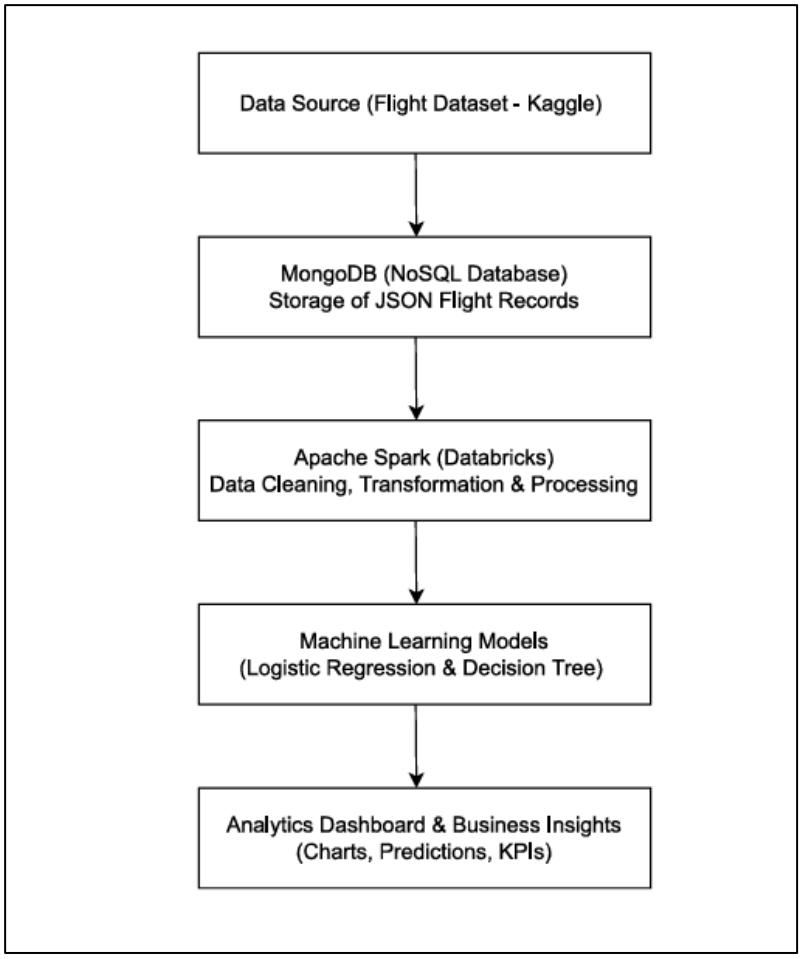

Figure 8.1: *Big Data Architecture for Flight Delay Prediction System*

The proposed architecture represents a complete big data workflow, integrating data collection, storage, distributed processing, machine learning, and visualization to generate meaningful insights for effective decision-making.

8.2 Component Descriptions

The architecture begins with the data source, which consists of airline on-time performance data obtained from Kaggle. This dataset contains large-scale flight records used for flight delay analysis and prediction.

MongoDB is used as a NoSQL database to store flight data in JSON format. This enables flexible handling of semi-structured data and demonstrates scalability in managing large datasets.

Apache Spark, implemented through Databricks, is used for distributed data processing. It performs data cleaning, transformation, feature engineering, and aggregation efficiently, ensuring high-performance analysis on large volumes of data.

Following preprocessing, the data is utilized to train machine learning models, specifically Logistic Regression and Decision Tree classifiers, to predict flight delays.

Finally, the results are presented through visualizations such as charts, confusion matrices, and evaluation metrics. These insights support improved operational efficiency and effective delay management in the airline industry.

9. Conclusion

This project demonstrated the practical application of big data analytics techniques on a large-scale airline dataset using Apache Spark and MongoDB. The analysis identified key delay patterns, contributing factors, and seasonal trends affecting airline performance. Machine learning models were developed to predict delays, with Logistic Regression showing better performance compared to Decision Tree. The use of MongoDB illustrated how NoSQL databases can efficiently store and manage large datasets. It also shows how modern big data tools can support large-scale data processing, analysis, and prediction, providing valuable insights for real-world applications in the aviation industry.

Future work could extend this system by establishing a live connection between MongoDB Atlas and Databricks using the MongoDB Spark Connector, enabling real-time ingestion of flight records directly into Spark for continuous processing and prediction. Additional improvements could include integrating real-time weather and air traffic APIs, applying ensemble models such as Random Forest or Gradient Boosting to improve prediction accuracy, and deploying the trained model as a REST API for operational use.

References

Apache Software Foundation (2024) *Apache Spark documentation*. Available at: https://spark.apache.org/docs/latest/ (Accessed: 6 April 2026).

Bureau of Transportation Statistics (2024) *Airline on-time performance data*. Available at: https://www.transtats.bts.gov (Accessed: 6 April 2026).

Databricks (2024) *Databricks documentation*. Available at: https://docs.databricks.com/ (Accessed: 6 April 2026).

Géron, A. (2022) *Hands-on machine learning with scikit-learn, Keras, and TensorFlow*. 3rd edn. O'Reilly Media.

Han, J., Kamber, M. and Pei, J. (2012) *Data mining: concepts and techniques*. 3rd edn. Morgan Kaufmann.

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An introduction to statistical learning*. 2nd edn. Springer.

Kaggle (2024) *Airline on-time performance data*. Available at: https://www.kaggle.com/datasets/bulter22/airline-data (Accessed: 6 April 2026).

MongoDB Inc. (2023) *What is NoSQL?* Available at: https://www.mongodb.com/nosql-explained (Accessed: 6 April 2026).

MongoDB Inc. (2024) *MongoDB Atlas documentation*. Available at: https://www.mongodb.com/docs/atlas/ (Accessed: 6 April 2026).

Provost, F. and Fawcett, T. (2013) *Data science for business*. O'Reilly Media.

Zaharia, M. et al. (2016) 'Apache Spark: a unified engine for big data processing', *Communications of the ACM*, 59(11), pp. 56–65.